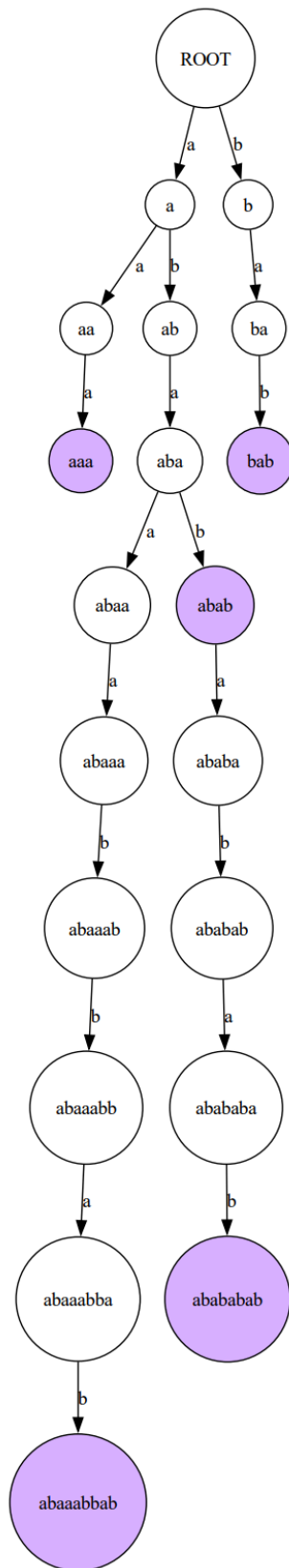


# **Algorithm for Bioinformatics Exercise 8**

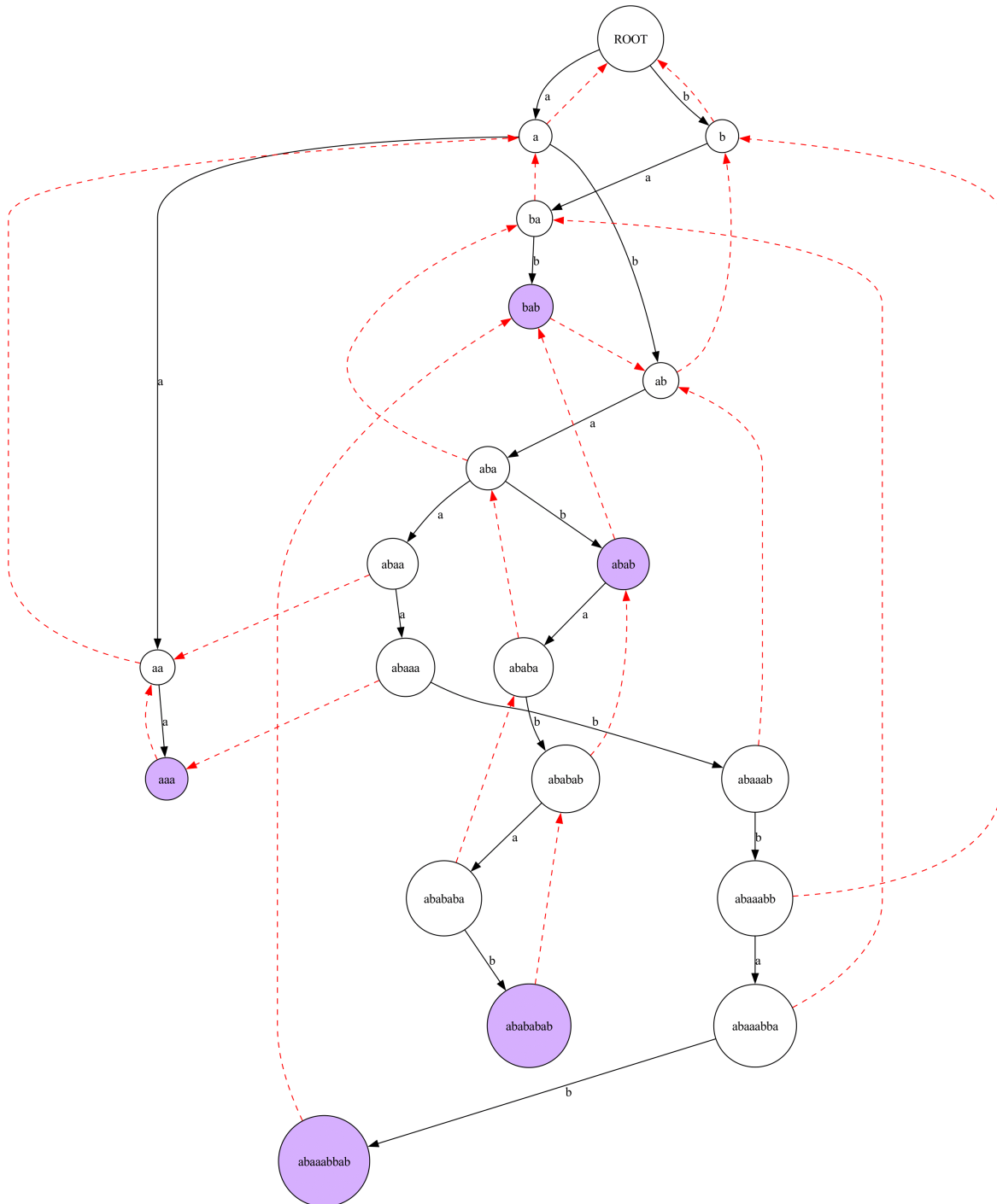
## **David Riemer, Usama Mehmood**

### **Problem 1**

The Automaton looks like this:



We now add the failure links:



## Problem 2

When building the algorithm, we have 2 steps, first we want to find all the occurrences of our dictionary  $S_1 \dots S_d$  of strings, in  $S$  ( $|S| = N$ ). Then we need to find the optimal alignment of the dictionary, minimizing the number of extra needed bits. We see this as maximizing the covered length in the dictionary.

We let  $L$  be the total length of positions in  $S$  that are covered by chosen dictionary occurrences (non-overlapping) and  $C$  be the cost.

Since every position of  $S$  is either covered (inside some occurrence) or uncovered (by adding a bit):

$$C = N - L$$

So, minimizing  $C$  is equivalent to maximizing  $L$ .

We preprocess the dictionary, by building the Aho-Corasick Automaton. We build a tree that has every dictionary string and:

- Each edge is labeled by 0 or 1.
- Each node represents a prefix of some dictionary string.
- Mark nodes that correspond to full dictionary strings  $S_j$ .
- Add the failure links

We then do a linear scan of the query string  $S$ : For every state  $q_i$  and all states reachable by following failure links we report all patterns that end at position  $i$  of the form :  $S_j$  occurs as

$$S[i - |S_j| + 1 \dots i]$$

We store the matches in a List of Integer Pairs,  $Matches[i]$  of length  $L$ , where every entry is the Id of the dictionary string, as well as its length, if it ends at position  $L$ .

We then want to decide, which patterns to pick and which bits to leave uncovered. We do this by defining an array  $maxCovered[]$ , with length  $L$ . We start by initializing  $maxCovered[0] = 0$ .

For every step in the query String  $S$  we consider leaving  $S[i]$  uncovered. In this case we set

$$maxCovered[i] = maxCovered[i - 1]$$

If we decide to cover a distance by a dictionary string  $S_j$  of length  $L_j$ , that means that the positions  $i - L_j + 1 \dots i$  are covered by the string. And in the positions  $i - L_j$  and before we should already have a optimal solution, that maximizes  $maxCovered[]$ . In this case we update the array with:

$$maxCovered[i] \geq maxCovered[i - L_j] + L_j$$

So for each  $i = 1 \dots N$  our array will be filled with:

$$maxCovered[i] = \max(maxCovered[i - 1], \max_{Patterns S_j \text{ ending at } i} (maxCovered[i - |S_j|] + |S_j|))$$

we then have our maximum cost as  $C = L - maxCovered[N]$ .